Assignment 02: Containerized Web Application Hosting, Load Balancing, and Automatic Scaling

Milestone 01: Develop a Sample Web Application

Develop a quadratic equation solver web application using HTML, jQuery, Bootstrap, and your choice of server-side technology such as PHP, Rails, Django, Spring MVC, etc. Instead of accepting the values of a, b, and c from the user, implement a server-side function that randomly generates three numbers `a`, `b`, and `c` each time it is called, ensuring that $a \neq 0$. The function should use these values to form and solve the quadratic equation $ax^2 + bx + c = 0$ and return the generated values of `a`, `b`, and `c`, along with the computed solution(s). The application should handle cases where the equation has two real solutions, one real solution, or no real solutions. In addition, the function must return the IP address and/or hostname of the server hosting the application.

The deliverable of this milestone is to provide a snapshot of your user interface with an output to some sample input and source code of your server-side implementation.

Milestone 02: Host your Web Application in a Docker Container

Now in this part of the assignment, you have to prepare an image of a Docker container containing your web application. Then run two instances of containers using the image your prepared. You must be able to use your Web application from your host operating system's browser. Each application container must be configured with a maximum resource limit of **1 GB of memory** and **1 CPU core**. These resource limits must be applied to both application containers.

The deliverable of this milestone is to provide a snapshot of the browser showing the use of the application and a list of running containers from the command prompt (`docker ps`).

Milestone 03: Deploy Nginx as a Proxy Server in a Docker Container

Now, you need to prepare another Docker container image containing Nginx. You need to use Nginx as a proxy server that will load balance the incoming traffic to your deployed containers hosting your web application. Remember! you deployed your application in two Docker containers as part of Milestone 02.

The deliverable of this milestone is to provide a snapshot of the browser showing the use of the proxy server to access your application, configuration to use Nginx as a proxy server and load balancer, and list of running containers from the command prompt.

Milestone 04: Metrics Collection and Autoscaling

In this milestone, you will develop an **autoscaler** that continuously monitors your application containers and automatically adjusts the number of containers according to the workload. The autoscaler should run as a standalone program on the **host machine** and should work with the application and Nginx containers developed in the previous milestones.

Part A: Metrics Collection

The autoscaler should periodically collect the following information:

1. Number of running application containers.
2. CPU usage of each application container.
3. Memory usage of each application container.
4. Average CPU and memory usage of all application containers.

The collected information should be stored in a file named `metrics.csv`.
For example:

```
timestamp,container_count,avg_cpu,avg_memory
06-01-2026 09:30:55,2,48.2,21.5
06-01-2026 09:31:25,2,52.7,22.1
```

Part B: Scaling Policy

Based on the collected metrics, your autoscaler should automatically scale the application.

- **Scale Out:** If the average CPU usage remains above **70%**, increase the number of application containers.
- **Scale In:** If the average CPU usage remains below **30%**, decrease the number of application containers.
- The number of application containers must always remain between a configurable **minimum of 1** and **maximum of 4**.
- After a scaling operation, the autoscaler should wait for a configurable **cooldown period** (e.g., 60 seconds) before performing another scaling operation.

When removing a container, it should first be removed from the load-balancing process. Existing requests should be allowed to complete before the container is stopped and removed.

Part C: Nginx Integration

The autoscaler should automatically keep Nginx updated whenever application containers are added or removed.

Your system should:

1. Automatically update the Nginx upstream configuration when application containers are added or removed.
2. Ensure that Nginx sends requests only to available and healthy application containers.
3. Reload Nginx after an upstream configuration change.
4. Ensure that users can continue accessing the application while containers are being added or removed.

The deliverables of this milestone include the source code of the autoscaler, the `metrics.csv` file containing sample collected metrics, the Nginx configuration used for load balancing, screenshots demonstrating both scale-out and scale-in operations, a screenshot showing the running application containers, and a short report explaining the working of the autoscaler and its scaling decisions.

Milestone 05: Synthetic Workload Generation and Autoscaling Evaluation

In this part of the assignment, you will use `httperf` to generate a synthetic workload for your web application. The workload must be sent to the **Nginx container**, which will distribute the requests among the application containers. The generated workload should be sufficient to demonstrate the automatic scaling behavior developed in Milestone 04.

You will run a series of experiments by gradually increasing and decreasing the workload. During each experiment, record the workload generated by `httperf`, the application's response time, CPU utilization, and the number of running application containers.

You should also use the CPU profiling utility developed as part of the assignment to monitor the CPU utilization during the experiment. The collected data should be used to analyze how the application's performance and number of containers change as the workload increases or decreases.

The deliverable of this milestone is to provide graphs showing (1) workload/request rate, (2) average response time, (3) CPU utilization, and (4) number of running application containers over time. You must also provide the output of `httperf` and a brief discussion explaining how the change in workload affected CPU utilization, response time, and the autoscaler's scale-out and scale-in decisions.

Submission Guidelines

You need to prepare a small report in PDF format containing deliverables for all the milestones before **October 14, 2026, 11:59 pm via Google Classroom**. Also, submit a loom video URL (maximum 4 minutes) showing all the relevant steps and working.